

Capital One: AI-Enabled Photography and Visual Brand Standards Analysis

Design Document – VT Capstone Group 4

Architectural Components

User Interface – Entry point for reviewers. It lets a user upload photographs or submit a website URL, select which brand standards to evaluate against, view the resulting dashboard, and mark agreement or disagreement with the AI's reasoning on a given image. It does not perform analysis; it collects user input, submits requests, monitors job status, and presents results. It is a React/TypeScript application that communicates only with the Backend API, including sending reviewer feedback to be saved.

Backend API – The coordinator. The User Interface communicates only with the Backend API, which invokes the other system components as needed. It receives uploaded photographs or a website URL and the selected evaluation criteria from the UI. For a URL, it starts the Web Crawler and routes the discovered images to the Image Preprocessor; uploaded photographs are sent directly to preprocessing. Because crawling and model inference may take several minutes, the API creates a background analysis job and returns a job identifier that the UI uses to request status updates. The background job coordinates preprocessing, computer-vision analysis, model inference, and scoring. The API also reads and writes the Data Storage layer, assembles completed results for the dashboard, and manages image-level errors so that one failed image does not terminate the entire batch.

Web Crawler – Given a URL, traverses a bounded set of pages rather than the entire domain and identifies image assets on each page. It owns page traversal and asset discovery, nothing more. It returns the discovered image URLs to the Backend API, which routes them to the Image Preprocessor. The crawler uses Playwright for JavaScript-rendered pages and BeautifulSoup for static pages.

Image Preprocessor – Filters out non-photographic assets (logos, icons, UI graphics) and standardizes the remaining photographs (format, size) so they're consistent inputs for analysis. This is the component that decides what counts as "a photograph worth evaluating", which matters because the crawler alone can't distinguish a hero image from a nav icon.

Computer Vision Analyzer – Measures quantifiable photographic attributes that can be calculated directly, including brightness, dominant colors, contrast, and framing. It uses OpenCV, Pillow, NumPy, and scikit-learn and returns structured measurements to the Backend API. Context-dependent judgments, such as tone, approachability, and brand fit, are handled separately by the multimodal model through the Model Serving component.

Model Serving – Hosts the selected open-weight vision-language model behind an HTTP endpoint. The Backend API sends it an image, and a prompt built from the selected criteria, and it returns structured JSON. The specific model and size will be finalized after benchmarking candidate models for accuracy, consistency, interpretability, compute requirements, and licensing. In deployment, the model may run on a dedicated GPU host, while a smaller compatible model may be used locally during development. Because the rest of the system only knows the endpoint, the model can be swapped without changing other components.

Scoring Engine - Validates and combines the objective measurements from the Computer Vision Analyzer with the qualitative judgments and reasoning returned by Gemma 4. It applies the weights from the selected rubric (loaded from JSON configuration files) to calculate criterion-level and overall scores, then formats the model's evidence into a human-readable explanation.

Data Storage – Persists batch results, criterion-level scores and explanations, and reviewer agree/disagree feedback in a relational database, and caches downloaded and preprocessed images in file storage. Only the Backend API reads from or writes to it.

Dashboard - A module within the React user interface that displays batch-level results returned by the Backend API, including overall patterns, score distributions, inconsistencies, flagged images, and individual explanations.

External Systems – The websites being crawled (accessed read-only) and Hugging Face Hub (used to download model weights during setup). No paid third-party APIs are used; all model inference is self-hosted.

User Device - The hardware the reviewer is sitting at (laptop, phone). Everything on the device side is just a browser rendering the UI; the actual work happens server-side.

These components form two input paths behind a coordinating API. Uploaded photographs proceed directly to preprocessing, while a submitted website URL first passes through the Web Crawler. Both paths then follow the same preprocessing, computer-vision analysis, multimodal analysis, scoring, explanation, storage, and display process. Because each component has a defined responsibility and communicates through the Backend API, the model or scoring criteria can be replaced without redesigning the complete system. This modularity allows the tool to support other organizations' brand standards rather than being tied specifically to Capital One's guidelines.

Commented [BS1]: Lets not lock this in design document, unless we ar fully committed.. but might need a bit of review and selection. <Gemma 4>. Made it more generalized.

Resources

Resource	How Used	Explanation
Python 3.11+	Primary language for backend, image preprocessing, and AI/scoring logic	Best-supported language for CV, ML tooling, open-weight model inference
FastAPI	Backend API framework, coordinates all component calls	Lightweight, async-friendly, and generates OpenAPI docs automatically, which matters for us
React	Frontend framework for UI	Provides a production-quality frontend.
OpenCV	Computer vision library for quantifiable image attributes (brightness, color distribution, contrast, framing)	Open-source, well documented, and mature. Handles analysis without needing a model call
BeautifulSoup / Playwright	Webcrawler, extract image URL from bounded set of pages	BeautifulSoup is good for static pages, Playwright handles JS-rendered pages so we can extract images from dynamic sites.
Candidate open-weight VLM: Gemma 4 family (31B primary, E4B for dev; final variant selected after benchmarking)	Judges tone, approachability, brand fit, and writes reasoning	Open-weight (Apache 2.0) and named as an approved option in Capital One's pitch. Requires dedicated GPU compute to host
Pillow	Image Preprocessor: converts formats and resizes photographs to consistent inputs	Standard open-source Python imaging library, works directly with the rest of the Python stack
NumPy	Array operations underlying image measurements and scoring math	Required by OpenCV and scikit-learn, so it's already in the stack
scikit-learn	K-means clustering to extract dominant colors from each image	Open-source and pairs with OpenCV, so dominant color needs no model call
Rule-based photo filter (custom Python module)	Image Preprocessor: excludes logos, icons, and UI graphics using dimensions, aspect ratio, and file type	Cheap first-pass filter; the crawler can't tell a hero image from a nav icon, so this component decides

Resource	How Used	Explanation
httpx	Downloads image assets the crawler finds	Async HTTP client that fits FastAPI's async design
Uvicorn	Runs the FastAPI application	The ASGI server FastAPI is built to run on
Pydantic	Validates API payloads and the model's JSON output before scoring	Forces consistent criterion scores and explanations instead of free-form text
vLLM	Serves Gemma 4 behind an HTTP endpoint the backend calls	Open-source model server, keeps the model swappable per the modularity requirement
Ollama	Runs Gemma 4 (E4B) locally during development	Easy local setup, so team members can test without the GPU server
Hugging Face Hub	Source for downloading Gemma 4 weights	Standard distribution point for open-weight models, keeps setup reproducible
Cloud GPU Host (AWS EC2)	Runs the production vision-language model through vLLM and exposes a private HTTP endpoint to the Backend API	Provides the dedicated GPU memory and compute required for larger models. The exact provider and instance type will be selected after benchmarking model size, performance, and cost.
SQLite + SQLAlchemy	Stores batches, scores, explanations, and reviewer agree/disagree feedback	Serverless and simple for a prototype; SQLAlchemy lets us move to PostgreSQL later
Local file storage	Caches downloaded and preprocessed images	Sufficient for a one-semester prototype, no extra service to run
TypeScript	Language for the React frontend	Catches type errors in the frontend and API data shapes
Node.js 24 LTS + Vite 8	Frontend build tooling and dev server	Fast builds and hot reload for React development
Tailwind CSS	Styling for UI components	Consistent look without writing custom CSS
Recharts	Dashboard charts (score distributions, flagged images)	Open-source and built for React

Commented [DP2]: I think this one also goes hand in hand with the web scraping portion, unsure if we need even need to web scrape...

Commented [DP2R2]: probably will, perhaps just leave it here

Commented [DP4]: Unsure if this is what we want to use for data storage

Commented [DP4R2]: I have used SQLAlchemy in a project before, but SQLite could be good for now just to keep things simple

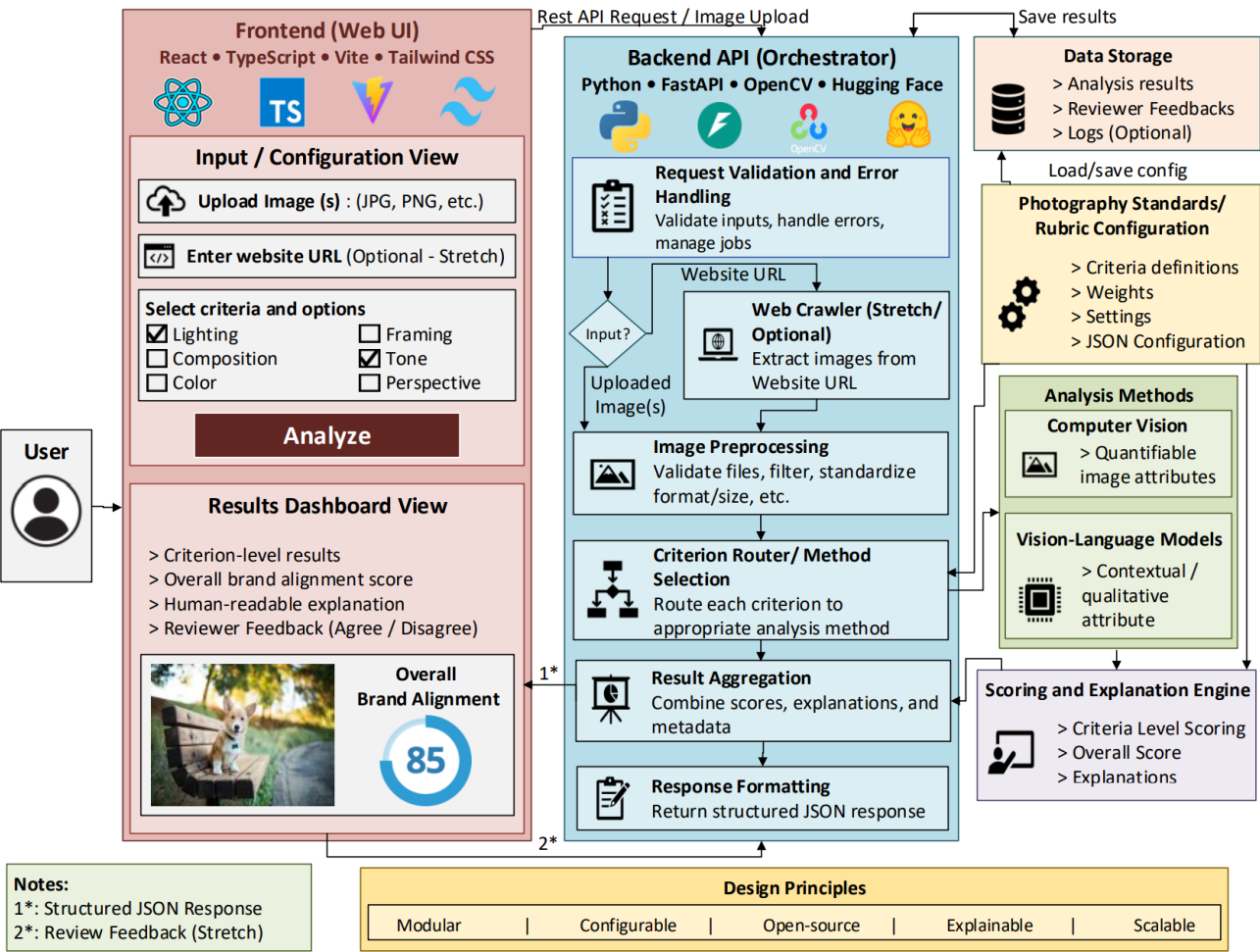
Commented [DP5]: we don't HAVE to use this but like if we're vibe coding the frontend which we probably will.....then idrc

Resource	How Used	Explanation
Docker + Docker Compose	Packages frontend, API, and model server	Reproducible setup for Capital One and other adopters
pytest	Unit tests for scoring, preprocessing, and API	Standard Python testing, supports the modular design
pip-licenses	Audits the license of every Python dependency	Capital One requires all packages to be open source
Test images: publicly released advertising/brand photography from Capital One and other organizations	Data for testing and evaluating the pipeline	Pitch allows only publicly released, approved ads; no internal Capital One data
JSON config files	Stores the rubric definitions (criteria names, descriptions, weighting) that the Scoring Engine evaluates against	Keeps the rubric decoupled from code, so a different org can swap in their own brand standards without touching app logic
GitHub	Version control and code hosting	We can review progress and have open-source publication
Apache 2.0 license	Licensing for final codebase	Open-source licenses; Gemma 4 is also Apache 2.0
User's own device	Client hardware, runs UI	No special hardware needed client side, all analysis happens server-side

AI Usage Statement:

GenAI was used to support research and discussion of system architecture, technical resources, usability, and design alternatives, and to review team-written text for clarity and grammar. The team independently reviewed and verified all submitted design decisions and technical information. Conversation logs are included in the appendix.

Architectural Design



Design Sketches

Screen 1 is where a reviewer adds photos or a URL and picks the brand standards.

1.Add photographs

Drop photos here

Or enter a website url

https://www.example.com

Pages to scan

10

2. Select brand Standards

Measured Attributes

☐ Lighting

☐ Framing and composition

☐ Focal Point

☐ Color palette

☐ Perspective and depth of field

Tone and message

☐ Approachable

☐ Confident

☐ Innovative

Rubric

Default photography standards

Analyze

Screen 2 displays the progress of the analysis job.

Analyzing photographs

Job started from campaign1.zip 42 images queued

Processing 14 of 23 images

~3 min remaining

☒ Preprocessing & filtering images

Done

☒ Running computer- vision analysis

Done

☐ Scoring subjective criteria (model inference)

In progress

☐ Aggregating scores and building dashboard

Waiting

You can leave this page open the job is running in the background

Cancel Job

Screen 3 opens when a reviewer selects an image. It displays criterion-level scores and explanations and allows the reviewer to provide Agree/Disagree feedback.

Batch results

Brand check complete - Default photography standards

campaign1.zip - 42 imgs

Avg alignment

68%

Needs review

11

of 42 images

Strong pass

24

of 42 images

Reviewed

7

of 42 images checked

AllNeeds reviewPassFail

Sort: Lowest first

social-campaign-bg.png

39%

Fail

storefront-exterior.jpg

55%

Needs review

card-lifestyle.jpg

61%

Needs review

team-photo-hq.jpg

72%

Pass

product-hero-v2.jpg

84%

Pass

app-mockup-device.png

91%

Pass

+ New analysis

Export report

card-lifestyle.jpg

Found on example.com/cards

Overall alignment

61%

Needs review

Lighting

Measured

4/5

Even exposure with soft shadows. Brightness and contrast fall within the target range.

Agree

Disagree

Color palette

Measured

2/5

Dominant colors skew cool and desaturated compared with the warm brand palette.

Agree

Disagree

Approachable

AI judgment

3/5

The subject's open posture feels welcoming, but the distant framing reduces warmth.

Agree

Disagree

Add a comment for the team

Save feedback

Last is the flow map, which shows how a reviewer moves between the screens.

